

Assignment 2

NAND Gate Implemented on Zynq

Marium Waleed Mostafa

August 2026

1 Objective

The objective of this assignment is to implement a NAND gate on the ZedBoard by designing an AND gate in hardware and using a C program running on the Zynq processing system to perform the logical inversion, producing the final NAND output.

2 Hardware Design

2.1 AND Gate RTL Design

A simple 2-input AND gate was written in Verilog and added as a design source.

```

22 :
23 module and_gate(
24     input a,b,
25     output c
26 );
27
28 assign c = a & b;
29 endmodule
30

```

Figure 1: AND gate Verilog source code

2.2 Block Design

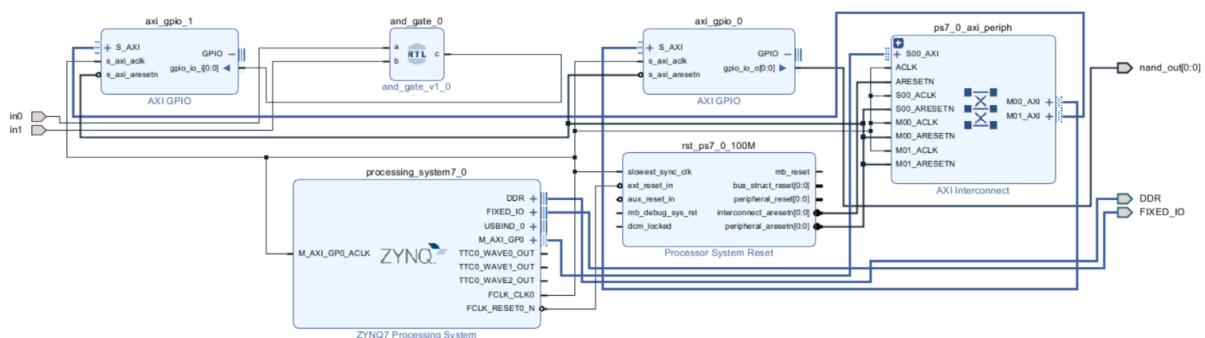


Figure 2: Complete block design: AND gate, AXI GPIOs, ZYNQ7 processing system, and AXI interconnect

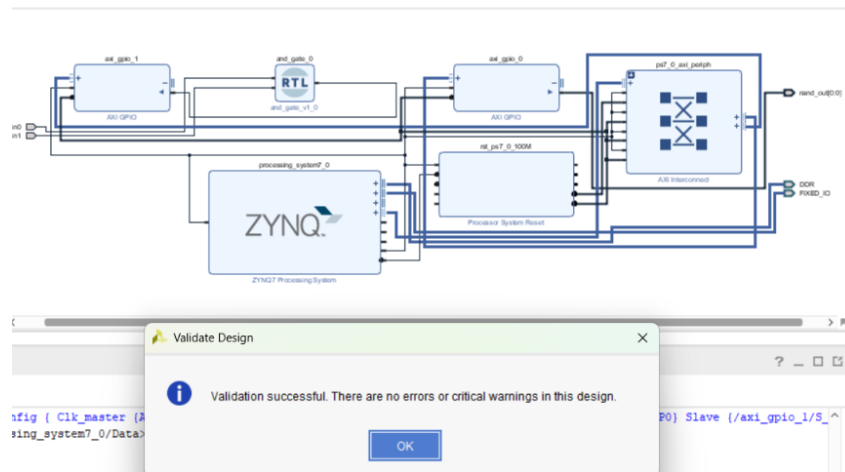


Figure 3: Successful block design validation

2.3 HDL Wrapper and Constraints

```

171
172 # -----
173 # User LEDs - Bank 33
174 # -----
175 set_property PACKAGE_PIN T22 [get_ports {nand_out}]; # "LDO"
176
177 ## -----
178 ## User DIP Switches - Bank 35
179 ## -----
180 set_property PACKAGE_PIN F22 [get_ports {in0}]; # "SW0"
181 set_property PACKAGE_PIN G22 [get_ports {in1}]; # "SW1"
182
183
184 # Note that the bank voltage for IO Bank 33 is fixed to 3.3V on ZedBoard.
185 set_property IOSTANDARD LVCMOS33 [get_ports -of_objects [get_iobanks 33]];
186 set_property IOSTANDARD LVCMOS18 [get_ports -of_objects [get_iobanks 35]];
187
188 # Note that the bank voltage for IO Bank 13 is fixed to 3.3V on ZedBoard.
189 set_property IOSTANDARD LVCMOS33 [get_ports -of_objects [get_iobanks 13]];

```

Figure 4: Pin and I/O standard constraints (.xdc file)

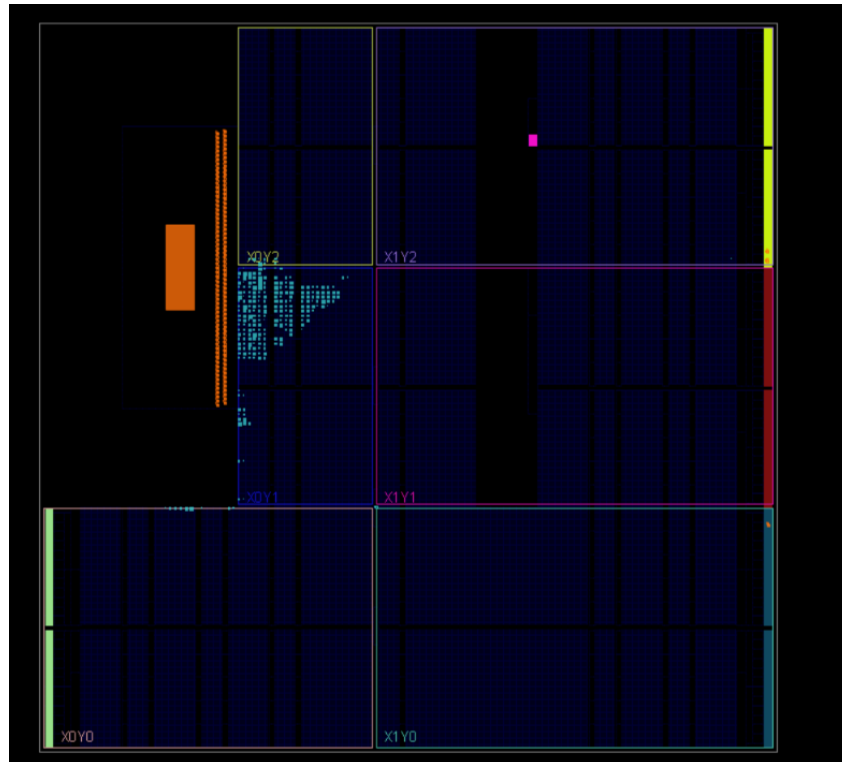


Figure 5: Implemented Design

3 Software Design (SDK)

3.1 C Code

```
#include <stdio.h>
#include "platform.h"
#include "xil_printf.h"
#include "xgpio.h"
#include "xparameters.h"

int main()
{
    init_platform();

    XGpio input, output;
    int a;
    int y;

    XGpio_Initialize(&input, XPAR_AXI_GPIO_0_DEVICE_ID);
    XGpio_Initialize(&output, XPAR_AXI_GPIO_1_DEVICE_ID);

    XGpio_SetDataDirection(&input, 1, 1);
    XGpio_SetDataDirection(&output, 1, 0);

    while (1) {
        a = XGpio_DiscreteRead(&input, 1);
        y = ~a;
        XGpio_DiscreteWrite(&output, 1, y);
    }

    cleanup_platform();
    return 0;
}
```

Figure 6: NAND inversion C code in the SDK editor

```
1
2 #include <stdio.h>
3 #include "platform.h"
4 #include "xil_printf.h"
5 #include "xgpio.h"
6 #include "xparameters.h"
7
8 int main()
9 {
10     init_platform();
11
12     XGpio input, output;
13     int a;
14     int y;
15     XGpio_Initialize(&input, XPAR_AXI_GPIO_0_DEVICE_ID);
16     XGpio_Initialize(&output, XPAR_AXI_GPIO_1_DEVICE_ID);
17     XGpio_SetDataDirection(&input, 1, 1);
18     XGpio_SetDataDirection(&output, 1, 0);
19
20     while (1) {
21         a = XGpio_DiscreteRead(&input, 1);
22         y = ~a;
23         XGpio_DiscreteWrite(&output, 1, y);
24     }
25
26     cleanup_platform();
27     return 0;
28 }
```

Figure 7: NAND inversion C code (alternate editor view)

3.2 Build

```
make: Nothing to be done for `main-build'.
08:58:38 Build Finished (took 157ms)
```

Figure 8: Successful build of the NAND application project